



CASTOR

Cphora · ERP/CRM/Contabilidad

Brief técnico de entrega

Paquete consolidado para programador full-stack senior

Cliente:	Castor SAS (fábrica de muebles, Colombia)
Producto:	Castor · Cphora — ERP/CRM/Contabilidad PUC integrado
Fecha:	2026-05-21 (v1.1 con correcciones)
Versión:	MVP v1.1
Documentos incluidos:	4 documentos técnicos + anexo de 13 archivos adjuntos

Este PDF consolida los requerimientos, manual del motor contable, changelog y guía de despliegue. Los archivos código (HTML, JS, SQL) se entregan por separado — ver anexo final.

Tabla de contenido

Portada	1
Tabla de contenido	2
Doc 1 · Requerimientos del MVP	3
Doc 2 · Manual técnico del Motor Contable PUC	—
Doc 3 · Changelog del Motor Contable	—
Doc 4 · Guía de despliegue a Supabase	—
Anexo · Archivos adjuntos a enviar por separado	—

Las páginas exactas de cada documento se identifican por el header de cada página (esquina superior derecha indica el doc activo).

Nota v1.1: Este brief incorpora las correcciones del 2026-05-21 sobre v1.0. Cambios principales: bodegas separadas PT/MP (§7.5), cuentas bancarias por defecto actualizadas con CRUD (§7.8), nómina totalmente editable (§7.11), categoría 'Otros' en activos fijos (§7.12), FomPlus como único exportador (§7.15), y reorganización completa de roles y permisos (§9).

1

Requerimientos del MVP

Castor · Requerimientos para MVP funcional

Versión: 1.1 · **Fecha:** 2026-05-21 · **Cliente:** Castor SAS (fábrica de muebles, Colombia) · **Producto:** Castor · Cphora — ERP/CRM/Contabilidad integrado · **Audiencia:** programador full-stack senior que tomará el proyecto

1. Resumen ejecutivo

Castor SAS necesita un sistema operativo único que cubra todo su ciclo de negocio — desde la captación del lead comercial hasta el cierre fiscal anual — operable por la propia fábrica sin contador externo en el día a día. Hoy existe un **prototipo HTML funcional** (Demo6) que demuestra el modelo de datos, los flujos y la lógica contable completa contra Plan Único de Cuentas colombiano. **El MVP consiste en llevar ese prototipo a una aplicación de producción** con backend real (Supabase), autenticación, multi-usuario, persistencia confiable y deploy automatizado, sin perder ninguna de las funcionalidades del demo.

El motor contable PUC (7.259 líneas, 40 tests pasando) y el schema/migrador Supabase **están terminados y validados**. Lo que falta es el frontend productivo y la capa de aplicación encima.

2. Contexto del negocio

2.1. Qué hace Castor

Fábrica colombiana de muebles de alta gama (Bogotá + Cartagena + Volcanes/Medellín). Vende por canales mixtos: web, showrooms (Castor 43, Castor Ctg, Volcanes), referidos, redes sociales. Produce a pedido (tipo: 'produccion') y mantiene stock disponible (tipo: 'stock'). Líneas: sillas, mesas, sofás, butacas, modulares, camas, complementos.

2.2. Flujo operativo

```
Lead (CRM) → Cotización → Pedido (cabecera) → OPs (1..N por producto)
  → Compra de insumos (OCs) → Recepción → Salidas a producción
  → Producción por estaciones (Ebanistería, Tapicería, etc.)
  → Producto terminado a CEDI → Reserva para entrega
  → Facturación/Remisión → Cobro → Garantías → Postventa
```

Cada uno de estos pasos dispara **un asiento contable automático** sobre el PUC colombiano (clases 1-7, ~120 cuentas auxiliares). Al final del mes la contadora externa puede exportar el libro completo (ver \$7.15).

2.3. Restricciones legales / normativa

- PUC colombiano (Plan Único de Cuentas) obligatorio en clases 1-7.
- DIAN: IVA 19/5/0, ReteFte (compras 2.5%, servicios 3.5%, honorarios 11%), ReteIVA 15%, ReteICA Bogotá 9.66×1000.
- Numeración interna NO oficial DIAN: las facturas/remisiones/NC/ND llevan numeración propia con leyenda "Numeración interna · No oficial DIAN". Para emisión válida a DIAN se conecta un proveedor tecnológico autorizado (Carvajal, FacturaPlus) — fuera del alcance del MVP.
- Nómina Colombia 2026: SMMLV \$1.423.500, aux transporte \$200.000, ARL III 0.522%, exoneración Ley 1607 (<10 SMMLV no carga 8.5% salud + 2% SENA + 3% ICBF), provisiones cesantías/intereses/prima/vacaciones.
- Moneda única: COP. No multimoneda.

3. Estado actual del proyecto

3.1. Artefactos entregados (en Cphora/)

Archivo	Líneas	Estado	Propósito
Castor_Dashboard_Demo6.html	8.778	Funcional	Prototipo completo del dashboard. Vanilla JS + Tailwind via CDN. Persiste en localStorage. Es la especificación visual y de UX del MVP.
castor_accounting.js	7.259	Funcional · 40/40 tests OK	Motor contable PUC: 13 hooks operativos, 6 procesos batch, ~80 funciones exportadas, exportador a FomPlus + CSV/JSON/XLSX. Reutilizable tal cual en el MVP.
supabase_schema.sql	750	Listo para aplicar	Schema base v1: 43 tablas (29 plataforma + 14 contables).
supabase_schema_v2.sql	~190	Listo para aplicar	Adiciones aditivas v2: columnas faltantes + tablas purchase_orders y purchase_order_items. Idempotente.
supabase_state_blob.sql	30	Listo para aplicar	Tabla auxiliar state_blobs para sync de respaldo.
_supabase_analytics_views.sql	~400	Listo para aplicar	8 vistas + 3 RPC para BI.
_migrate_to_supabase.js	~350	Funcional	Migrador Node que vuelca el seed real a SQL inserts en orden topológico FK.
_migration_castor.sql	577	Regenerable	SQL listo para pegar en Supabase SQL Editor.
_run_tests_node.js	—	40/40 OK	Runner Node de los 40 tests unitarios del motor contable.
_smoke_e2e.js	—	76 OK	Smoke E2E que ejercita el ciclo operativo completo.
_smoke_real_state.js	—	34 OK	Smoke contra el seed real para detectar regresiones de datos.
_smoke_browser.html	—	Funcional	Smoke ejecutable en el navegador.
README_ACCOUNTING.md	~400	Completo	Manual técnico del motor contable.
CHANGELOG_ACCOUNTING.md	~210	Completo	Historial de las 6 fases del motor contable + bugs arreglados.
DEPLOYMENT_GUIDE.md	~470	Completo	Guía paso a paso de deployment a Supabase en 4 fases.

3.2. Métricas técnicas verificables

Tests unitarios:	40/40 OK
Smoke E2E:	76 OK · 0 fallas
Smoke real-state:	34 OK · 0 fallas
Cuentas PUC seed:	198
Hooks operativos:	13 (todas las operaciones del ciclo)

```

Procesos batch:      6 (nómina, depreciación, cierres, NC/ND)
Funciones exportadas: ~80
Tablas Supabase:    46 (43 v1 + 2 v2 + state_blobs)
Exportador ERP:     1 (FomPlus) + 3 genéricos (CSV, JSON, XLSX)

```

3.3. Lo que YA está resuelto (no reinventar)

- Modelo de datos completo — ver Castor_Dashboard_Demo6.html función seed() (líneas 332-518) + estructuras post-migrateLegacy.
- Schema Supabase relacional — aplicar los 3 SQL en orden produce un schema listo.
- Motor contable PUC — castor_accounting.js es independiente y portable. Cárgalo como módulo en el MVP.
- Migración inicial de datos — node _migrate_to_supabase.js genera el SQL completo.
- Sincronización bidireccional con Supabase — Demo6 ya tiene _sbPush (blob), _sbPushDirtyJournals (estructurado por JE) y _sbPullJournals (refresh desde nube). El MVP debe extender estos patrones.
- Tests automáticos — los 40 unit tests + 3 smokes corren en Node sin navegador. Mantenerlos en CI.

3.4. Lo que falta construir (alcance del MVP)

- Frontend productivo (Next.js / Nuxt / Remix recomendado; no quedarse en HTML monolítico).
- Autenticación real con Supabase Auth.
- Row Level Security (RLS) por rol en todas las tablas.
- Realtime para colaboración multi-usuario.
- Storage para soportes de pago, comprobantes, fotos de garantía.
- CI/CD y hosting de producción.
- Backups automáticos y plan de recuperación ante desastres.
- Logging y monitoreo básicos.
- Documentación de operación (runbook).

4. Alcance del MVP

4.1. Módulos funcionales obligatorios

Todos los siguientes módulos del Demo6 deben quedar funcionales en el MVP. Cada uno con su UI correspondiente, sus formularios, sus listados, sus modales de detalle, sus slide-panels y sus hooks contables conectados.

Importante: la tabla siguiente es solo el índice. El inventario exhaustivo página por página — con cada KPI, filtro, acción, modal, slide-panel, transición de estado y hook contable de Demo6 — está en §15 (Apéndice — Inventario funcional exhaustivo Demo6). El programador debe usar §15 como checklist 1:1 contra Demo6, no esta tabla.

Módulo	Sub-vistas	Hook contable que dispara
Inicio (Dashboard)	KPIs del mes, alertas, accesos rápidos	—
CRM	Leads, seguimientos, clasificación, conversión a cliente	—

Módulo	Sub-vistas	Hook contable que dispara
Cotizaciones	Crear, enviar, aceptar, vencer; cálculo IVA inclusive	—
Clientes	Personas + institucionales; perfil tributario (retenedor, actividad)	—
Pedidos	Cabecera por cliente + agrupa OPs	—
OPs (Órdenes de Producción)	Por producto; trayectoria de estaciones	postFinishedGoods al cerrar
Productos	Catálogo, BOM (lista de materiales), categorías, fotos	—
Almacén Insumos	Stock por bodega, movimientos	postRawMaterialConsumption, postRawMaterialReturn
Almacén PT	finished_stock, reservas, despachos	postFinishedGoods
Proveedores	Datos, NIT, retenciones aplicables	—
OCs (Órdenes de Compra)	Crear, parcializar recepciones	postSupplyPurchase al recibir
Pagos clientes	Anticipos, totales, retenciones recibidas	postCustomerCollection, postCustomerAdvance
Pagos proveedores	Liquidación de CxP	postSupplierPayment
Bancos	Saldos, ajustes manuales, conciliación	postBankAdjustment
Facturación	Emisión factura/remisión, NC, ND	postSale, emitNotaCredito, emitNotaDebito
Garantías	Reporte, diagnóstico, costos, resolución	postWarrantyCost
Postventa	Llamadas, NPS, encuestas	—
Empleados	Datos, salarios, área	—
Nómina	Cálculo + contabilización mensual	runPayroll + postPayroll
Activos fijos	Catálogo, depreciación lineal	runMonthlyDepreciation + postDepreciation
Innovación	Pipeline de ideas / pilotos / productos nuevos	—
Auditoría	Findings internos por área + cierre/verificación de OPs	—
Marketing	Activos (catálogos, posts, reels), vinculados a productos	—
Notificaciones	Por usuario, leídas/no leídas	—
Contabilidad	Subsección con 25+ vistas (ver §4.2)	(todas las anteriores)

4.2. Subsección Contabilidad (sub-sidebar de 25+ vistas)

Cubre el motor contable PUC. Todas estas pantallas ya existen en Demo6, hay que portarlas:

- **Operación:** Dashboard contable, Libro diario, Libro mayor, Auxiliar de cuenta, Asientos manuales, Contabilidad de facturación.
- **Reportes:** Balance de prueba, Balance general, P&G, Cartera (aging), CxP (aging), Aux IVA, Aux Retenciones.
- **Procesos:** Nómina, Activos fijos, Cierres y reaperturas de periodo, Cierre fiscal anual.
- **Configuración:** Plan PUC (CRUD activación), Mapeos operación→cuenta, Reglas tributarias, Numeración FAC/REM/NC/ND, Centros de costo, Perfil tributario.
- **Exportación:** Hub único con selección de vista + formato (FomPlus, CSV, JSON, XLSX).
- **Calidad:** Dashboard de tests (ejecuta los 40 unit tests desde el navegador).

4.3. Lo que NO entra en el MVP

- Emisión electrónica DIAN (requiere proveedor tecnológico autorizado).
- Multimoneda.
- Multi-empresa (Castor es una sola entidad).
- Conciliación bancaria automática (queda manual vía ajustes).
- Renta presuntiva / declaración anual automática.
- CIF dinámico (drivers de costos indirectos automáticos).
- App móvil nativa (responsive web es suficiente).
- Integraciones con marketplaces, redes sociales, PSE, pasarelas de pago.

5. Arquitectura técnica sugerida

5.1. Stack recomendado

Capa	Tecnología	Por qué
Base de datos	Supabase (Postgres 15+)	Decidido. El schema y migrador ya apuntan ahí.
Auth	Supabase Auth	Integrado con RLS de Postgres.
Realtime	Supabase Realtime	Multi-usuario sin servidor adicional.
Storage	Supabase Storage	Buckets para soportes (pagos/, garantías/, marketing/).
Frontend	Next.js 14+ (App Router) / Nuxt 3 / Remix	A elegir por el programador. Necesario SSR o SSG para tiempos de carga buenos.
UI library	Tailwind CSS (ya usado) + shadcn/ui o Headless UI	Tailwind ya está en el demo, no reescribir estilos.
State client	Zustand o React Query / TanStack Query	El motor contable opera sobre state JS — el cliente lo refleja.
Charts	Chart.js (ya usado) o Recharts	KPIs y cash flow.
PDF	jsPDF (ya usado para libros internos) o react-pdf	Libros oficiales internos con marca de agua.

Capa	Tecnología	Por qué
Tests	Jest / Vitest + los 40 unit tests existentes adaptados	Mantener el motor contable testeado.
CI/CD	GitHub Actions	Tests + deploy automático.
Hosting	Vercel (Next), Netlify, Cloudflare Pages	Static + Edge Functions si hace falta.

El programador puede sugerir alternativas razonadas. Lo no negociable: **Supabase** como BD y **mantener el motor contable JS** (castor_accounting.js) sin reescribirlo en otro lenguaje.

5.2. Modelo de ejecución del motor contable

El motor (castor_accounting.js) opera sobre un objeto state en memoria con todas las colecciones. Hoy en el demo:

```
localStorage ↔ state (memoria) ↔ Supabase (vía _sbPush*/_sbPull*)
```

Para el MVP propongo:

```
Supabase (fuente de verdad) ↔ state cliente (memoria, hidratado en login)
                               ↔ cache local (IndexedDB opcional para offline)
```

Cada operación del usuario:

- Llama al hook contable correspondiente (postSale, postCustomerCollection, etc.) sobre el state cliente.
- El hook valida + genera el journalEntry + lo persiste en state.journalEntries/Lines.
- El cliente hace upsert a Supabase de la(s) fila(s) afectada(s) (patrón ya implementado en _sbPushDirtyJournals).
- Si la upsert falla, se marca dirty y se reintenta en el próximo save.

Los hooks contables son **idempotentes** ((source, sourceId) único) — re-emitirlos no duplica asientos.

5.3. Edge Functions / triggers en Supabase

Mínimo viable: ninguno. Toda la lógica corre cliente.

Recomendado (segunda iteración):

- Trigger SQL en journal_entries para auditoría (quién creó qué, cuándo).
- Edge Function para PDFs grandes (libros oficiales) que no convenga generar cliente.
- Edge Function para programar la corrida de nómina mensual automática.

6. Migración de datos

6.1. Orden de aplicación en Supabase

- Crear proyecto Supabase (Free tier es suficiente para el primer año).
- SQL Editor → New query → pegar y ejecutar en este orden:
Cphora/supabase_schema.sql — schema base
Cphora/supabase_schema_v2.sql — adiciones aditivas (idempotente)
Cphora/supabase_state_blob.sql — tabla de respaldo blob
Cphora/_supabase_analytics_views.sql — vistas + RPC para BI
- Backup del state del cliente (Demo6.html localStorage → JSON file).

- Generar migración: `node Cphora/_migrate_to_supabase.js` → produce `_migration_castor.sql` (577 líneas).
- Aplicar la migración en Supabase SQL Editor.
- Verificar con las queries de `DEPLOYMENT_GUIDE.md` §1.3 (cuentas por tabla, integridad FK, ecuación contable $A=P+Pat+Util$).

6.2. Migración de un state real (no del seed)

El migrador por defecto lee el seed embebido en Demo6. Para migrar **el state real del cliente** (Bogotá ya operando con datos reales):

- Exportar `localStorage` del navegador del usuario (instrucciones en `DEPLOYMENT_GUIDE.md` §0).
- Modificar `_migrate_to_supabase.js` para leer ese JSON en vez del seed.
- Correr el migrador y aplicar.

6.3. Migración incremental post-MVP

Cada nuevo asiento contable (post o reverso) se sincroniza estructuradamente vía `_sbPushDirtyJournals` (ya implementado). Para tablas no contables, replicar el patrón:

- Tener un `state._dirtyEntities = { customers: Set, products: Set, orders: Set, ... }`.
- En cada `save()`, recorrer las claves y hacer `upsert` de las filas marcadas.
- Cada hook (crear cliente, editar producto, etc.) marca su entidad como `dirty`.

Este trabajo de "Fase 3" de la migración está documentado en `DEPLOYMENT_GUIDE.md` pero no implementado para entidades no contables. **El MVP debe completarlo.**

7. Requerimientos funcionales — detalle por módulo

Cada módulo del MVP debe replicar el comportamiento exacto del Demo6. El programador debe consultar el HTML del demo y el seed () para entender shapes de datos, transiciones de estado y validaciones.

7.1. CRM / Leads

- Tipos: lead (persona) o institucional (empresa con razón social + NIT).
- Canales: Llamada, Página Web, Tienda 43, Instagram, WhatsApp, Volcanes Showroom, Castor Cartagena, Referidos.
- Estados: Nuevo → En gestión → Compro / No interesado.
- Clasificación: alto / medio / bajo (sirve para filtrar follow-ups).
- Productos de interés (`productosInteres[]`) con qty estimada.
- Seguimientos (`notes[]` y `seguimientos[]`) con timestamp + autor.
- Conversión a cliente: crea registro en `customers` con `linkedLeadId` cruzado.

7.2. Cotizaciones

- Items: `{productId, qty, price, desc}`.
- `discount %` global.
- `vigenciaDias` (default 30).
- Estados: borrador → enviada → aceptada / vencida.
- `vigencia` (fecha calculada).

- Aceptar cotización → crear pedido + OPs.

7.3. Pedidos y OPs (estructura V7)

- Pedido (cabecera): un cliente, una fecha, un quoteld opcional, agrupa N OPs.
- OP (orden de producción): un producto, una qty, un pedidold. Trayectoria por 9 estaciones de producción (Programación → Ebanistería → Preparación y pintura → Metal mecánica → Pintura electrostática → Tapicería → Tejido → Ensamble → Listo).
- Cada OP tiene su tipo: stock (de inventario) | produccion (a fabricar) | innovacion (piloto).
- Cuando una OP llega a "Listo" se entra a finished_stock (con postFinishedGoods).

7.4. Productos

- SKU obligatorio único.
- Categorías: Sillas, Mesas de comedor, Sofás, Butacas, Complementos, Camas, Nocheros, Escritorios, Exterior, Colchones, Accesorios, Tapetes, Otros.
- BOM: array de {supplyId, qty} — necesario para postFinishedGoods.
- Dimensiones (ancho/alto/profundidad).
- Estados ciclo de vida: Activo / Borrador / Proceso / Inactivo.
- verified boolean (controla si está disponible para vender).

7.5. Almacén (insumos + PT)

Corrección v1.1: separación estricta entre bodegas de Producto Terminado (PT) y Materia Prima (MP). Cada bodega maneja un único tipo de stock — los movimientos contables se enrutan por tipo de bodega.

- **6 bodegas con tipo asignado:**
 - **CEDI** → solo PT (Centro de Distribución)
 - **Castor 43** → solo PT (showroom Bogotá)
 - **Castor Ctg** → solo PT (showroom Cartagena)
 - **Volcanes** → solo PT (showroom Medellín)
 - **Insumos 1** → solo MP (materia prima)
 - **Insumos 2** → solo MP (materia prima)
- Validación: el formulario de movimientos/recepciones debe filtrar bodegas según el tipo de item (producto terminado vs insumo). No se permite mover MP a una bodega PT ni viceversa.
- Recepciones de OC → entra a supply_receipts (solo a Insumos 1 o Insumos 2).
- Salidas a producción → supply_issues (con orderIds[]) desde Insumos 1/2.
- PT entra a finished_stock (en CEDI, 43, Ctg o Volcanes) con status disponible/reservado/en_tránsito.
- Movimientos manuales (stockMoves) con razón + autor; validan compatibilidad de tipo de bodega.

7.6. Compras (OCs)

- Estado: abierta / parcial / cerrada / cancelada.
- Items: array {supplyId, qty, unit, cost, received, status}.
- Recibir parcialmente → updates items[i].received + estado.
- Al cerrar (full received) dispara postSupplyPurchase con IVA + retenciones aplicables al proveedor.

7.7. Pagos clientes y proveedores

- Cliente: anticipos (pre-factura) → postCustomerAdvance (CR 280505). Cobros contra factura → postCustomerCollection (DB banco / CR 130505, + retenciones recibidas si el cliente es retenedor).
- Proveedor: outgoing_payments con tipo:'proveedor'|'empleado'. Liquidación contra OC → postSupplierPayment.
- Métodos: Transferencia, Efectivo, Tarjeta, Link de pago.
- Soporte adjunto (filename + size + uploadedAt) — en MVP guardado en Supabase Storage.

7.8. Bancos

Corrección v1.1: nuevas cuentas bancarias por defecto en el seed y CRUD completo expuesto en la UI.

- **4 cuentas por defecto en el seed:** Bancolombia, BBVA, Bold, Lulo Bank.
- **CRUD completo desde la UI:** el usuario debe poder **registrar** nuevas cuentas, **editar** los datos de cuentas existentes (nombre, tipo, número, color, holder) y **eliminar** cuentas que no tengan movimiento. La acción "Eliminar" debe bloquearse si la cuenta tiene asientos contables asociados (saldo distinto de cero o movimientos históricos).
- Para cada cuenta nueva, ensureBankPucAccount crea automáticamente la auxiliar 1110-XX en el Plan PUC.
- Saldo computado: balance inicial + $\Sigma(\text{pagos in}) - \Sigma(\text{pagos out}) + \Sigma(\text{ajustes})$.
- Conciliación: ajustes manuales con concepto (postBankAdjustment → DB/CR 1110-XX vs 425095/539595, con banco como tercero placeholder).

7.9. Facturación / Remisión / NC / ND

- Tipos doc: factura (con IVA inclusive 19%) | remisión (sin IVA).
- Estados: borrador → emitida → entregada → pagada / vencida / anulada.
- Numeración interna (FAC/REM/NC/ND) con badge "No oficial DIAN".
- NC: total (anula factura + reverso COGS) o parcial (por items).
- ND: incrementa cobro.
- Si la factura está en periodo cerrado, NC/ND se contabiliza en periodo abierto vigente.

7.10. Garantías y postventa

- Garantías: motivo + causal + estado (abierto → diagnóstico → solución → cerrado).
- Costos: caja (CR 110510) o almacén (CR 140505) → DB 519540.
- Fotos en Supabase Storage (garantias/{id}).
- Postventa: llamada / encuesta / visita; NPS.

7.11. Nómina

Corrección v1.1: todos los parámetros de cálculo de nómina deben ser **totalmente editables** desde la UI de configuración (Contabilidad → Configuración → Perfil tributario → Nómina). Los valores PAYROLL_2026 son solo defaults iniciales; el usuario puede sobrescribirlos sin tocar código.

- 7 empleados por defecto (cargos ventas, taller, almacén, contabilidad).
- Cálculo mensual según parámetros editables (defaults PAYROLL_2026):
 - SMMLV: \$1.423.500 (**editable**)
 - Auxilio de transporte: \$200.000, aplica si salario < 2 SMMLV (**editable: monto y tope SMMLV**)
 - Salud emp 4% / empr 8.5% (exonera Ley 1607 <10 SMMLV) — **tasas y tope editables**

- Pensión emp 4% / empr 12% — **tasas editables**
- ARL III 0.522% — **tasa y nivel de riesgo editables**
- Parafiscales: SENA 2% + ICBF 3% + Caja 4% (=9%; exonera <10 SMMLV) — **tasas y tope editables**
- Provisiones cesantías 8.33% + int cesantías 1% + prima 8.33% + vacaciones 4.17% — **todas editables**
- Persistencia: los valores editados viven en `state.companyTaxProfile.payrollParams` y sobreviven recargas.
- Validación: cambios solo afectan corridas futuras de nómina; las corridas ya posteadas conservan sus parámetros vía `paramsSnapshot`.
- Contabilización por área: MOD (720505+720570) para producción; admin/ventas a 5105xx/5205xx.
- Run idempotente por `periodId`.

7.12. Activos fijos

Corrección v1.1: añadida la categoría **Otros** para activos que no encajan en las categorías predefinidas.

- Categorías: oficina, computación, transporte, maquinaria, muebles, **otros**.
- Línea recta, vida útil en meses.
- Run mensual idempotente.
- Cuentas según uso (admin: 516015/159220; ventas: 526010/159225; CIF: 730560/159240/159205). Para categoría "otros" usar mapeo configurable desde `renderConfigMapeos` (default: admin 516015/159220).

7.13. Reportes contables

Todos los reportes de Fase 5 ya implementados en `castor_accounting.js`:

- Balance de prueba (8 columnas: saldo inicial / movimientos / saldo final).
- Balance general (Activo / Pasivo / Patrimonio agrupado por subgrupo).
- P&G del periodo.
- Cartera y CxP (aging 0-30 / 31-60 / 61-90 / 91+).
- Aux IVA y Aux Retenciones con exportación Formulario 300.

7.14. Cierres

- Cierre mensual (`closePeriod`): traslada clases 4-7 a 360505.
- Reapertura (`reopenPeriod`): solo gerencia, requiere no haber periodo posterior cerrado.
- Cierre fiscal anual (`closeFiscalYear`): cascada de 12 meses + 360505 → 370505.
- Libros internos PDF con marca de agua "DOCUMENTO INTERNO · NO OFICIAL DIAN".

7.15. Exportadores externos

Corrección v1.1: se eliminan los exportadores Siigo, World Office, Helisa y Contapyme. **El único exportador ERP soportado es FomPlus.** Los exportadores genéricos (CSV, JSON, XLSX) se mantienen.

- **FomPlus:** formato propio de FomPlus para integración directa. Layout y headers según especificación oficial de FomPlus vigente (a confirmar con el proveedor antes de implementar el formato definitivo).
- CSV genérico: BOM UTF-8 separador coma.
- JSON genérico: array de objetos.

- XLSX genérico: BOM separador punto y coma (compatible Excel-CO).

La implementación previa de Siigo/WO/Helisa/Contapyme en `castor_accounting.js` puede dejarse en el código como código muerto o eliminarse — no se expone en la UI del Hub de Exportación.

8. Requerimientos no funcionales

8.1. Performance

- Carga inicial < 2s en conexión 4G colombiana.
- Cualquier operación (post factura, calcular nómina) < 500ms en cliente.
- Reportes con < 10.000 asientos deben renderizar < 1s.
- Volumen esperado: ~100 facturas/mes, ~10 años de historia → < 200 MB total. Cabe en Supabase Free.

8.2. Seguridad

- Auth: Supabase Auth con email/password. Recuperación de contraseña por email.
- RLS habilitada en TODAS las tablas. Política base: `authenticated` puede `SELECT` en todas.
- Roles contabilidad, admin, auditoria y tesoreria pueden `INSERT/UPDATE` en tablas contables según matriz §9.
- Roles comercial, produccion, almacen, marketing, innovacion, hr, subauditoria solo pueden tocar su área (ver §9).
- No credenciales en código fuente: usar variables de entorno (`.env.local` no commiteado).
- Auditoría: cada `journalEntry` lleva `createdBy` y `postedBy` con `auth.uid()`.

8.3. Disponibilidad y backups

- Supabase hace backup automático diario (Pro tier en adelante).
- Para MVP en Free tier: cron job semanal que exporte tablas críticas a Storage como dump SQL.
- Plan de recuperación: documentar restore desde backup en runbook.

8.4. Mobile / responsive

- Layout responsive (no app nativa).
- Vistas críticas (Inicio, OP, Facturación) deben ser usables en pantalla 768px+.
- Imprimir libros PDF debe funcionar en desktop (Chrome/Edge/Safari).

8.5. Internacionalización

No requerida. UI 100% español (es-CO). Moneda COP. Fechas dd/MM/yyyy.

8.6. Browser support

Chrome / Edge / Firefox / Safari últimas 2 versiones. IE: no.

9. Roles y permisos

Capítulo reescrito en v1.1: se añaden los roles de Marketing, Innovación, Almacén, Auditoría, Subauditoría, HR y el subrol Contabilidad-Tesorería; además se introduce el mecanismo de **permisos delegables** mediante el cual los Admins pueden otorgar permisos puntuales a cualquier usuario de cualquier rol.

9.1. Matriz de roles principales

Rol	Cerrar mes	Reabrir mes	Emitir NC/ND	Tocar config PUC	Correr nómina	Ve reportes...
admin / gerencia	Sí	Sí	Sí	Sí	Sí	Sí (todos)
contabilidad	Sí	No	Sí	Sí (config)	Sí	Sí (contables)
contabilidad · tesorería *	No	No	No	No (solo bancos/pagos)	No	Sí (cartera, CxP, bancos)
comercial	No	No	No	No	No	Solo cartera propia
produccion	No	No	No	No	No	No
almacen	No	No	No	No (solo insumos/OCs)	No	Solo inventario y OCs
auditoria	Sí (cierre OPs)	No	No	Sí (OPs, productos)	No	Sí (todos)
subauditoria	No	No	No	No (solo comentarios)	No	Sí (view-only)
marketing	No	No	No	No (solo marketing assets)	No	Solo dashboard marketing
innovacion	No	No	No	No (solo productos innov + BOM)	No	Solo pipeline innov
hr	No	No	No	No (solo empleados + nómina input)	No	Solo RRHH y nómina

(*) **tesorería** es un subrol de contabilidad: hereda los permisos contables de lectura pero su capacidad de escritura se restringe a bancos, pagos in/out y conciliación. Ver §9.3.

9.2. Detalle de roles nuevos

Marketing

- **Ámbito:** módulo Marketing (15.3.8) y todo lo relacionado con marca/comunicación.
- **Puede:** crear, editar y eliminar marketing assets (catálogos, fotos producto/ambiente/cliente, feed, historias, campañas); vincular assets a productos; subir archivos a Supabase Storage bucket marketing/.
- **Puede ver:** catálogo de productos (read-only), ficha de cliente (datos básicos), KPIs de leads por canal.
- **No puede:** tocar precios, costos, BOM, ni ningún módulo contable.

Innovación

- **Ámbito:** módulo Innovación (15.3.17) y catálogo de Productos.
- Puede: crear productos nuevos (estado Borrador → Proceso → Activo), **alimentar el BOM** (array de {supplyId, qty}), **marcar las estaciones de producción** por las que pasa cada producto (subset de las 9 estaciones del flujo OP), votar ideas, mover ideas en el kanban (idea → piloto → producción), crear productos desde una OP de innovación con foto adjunta.
- Puede ver: catálogo de insumos (para construir BOM), histórico de OPs de innovación.
- No puede: emitir facturas, tocar precios de venta, modificar productos ya verificados (eso pasa por Auditoría).

Almacén

- **Ámbito:** módulo Almacén (12.a–12.e) y proveedores.
- Puede: **crear órdenes de compra** (OCs) y enviarlas a proveedores; **registrar entradas de insumos** (recepciones de OC, con foto remisión); **registrar salidas de insumos** hacia producción (supply_issues con orderIds[]); **crear proveedores** (CRUD completo: nombre, NIT, contacto, retenciones aplicables); **crear insumos** (CRUD); registrar movimientos manuales de stock con razón.
- Puede ver: stock por bodega, histórico de OCs, OPs pendientes que requieren consumo de insumos.
- No puede: emitir facturas, tocar contabilidad, tocar productos terminados (PT es responsabilidad de CEDI/auditoría).
- Dispara hooks contables: postSupplyPurchase al cerrar OC, postRawMaterialConsumption al registrar salida, postRawMaterialReturn en devoluciones.

Auditoría

- **Ámbito:** control de calidad y cierre del ciclo operativo. Vinculado al módulo Auditoría (15.3.16) y a OPs/Productos.
- Puede: **asegurarse de que entre bien el pedido** (verificar items, cliente, dirección, anticipo antes de aceptar la venta); **crear y cerrar Órdenes de Producción** (incluyendo crear OP desde auditoría con auditCreateOp, aprobar/rechazar OPs en revisión, corregir OPs rechazadas, cerrar OPs cuando finaliza producción); **verificación de productos nuevos** (verifyProduct para marcar verified=true en productos creados desde innovación); **editar productos** (cualquier campo, incluyendo activación/desactivación de venta); **editar OPs** (cualquier campo de la OP con auditEditOrder); guardar consumos auditados; verificar OPs cerradas; crear productos en CEDI desde auditoría.
- Puede ver: todos los reportes contables, todas las facturas, todas las OPs.
- No puede: emitir NC/ND, tocar configuración PUC, correr nómina.
- Genera findings en state.audits[] con tipo (interna/externa), área, responsable y estado (abierta/parcial/cerrada).

Subauditoría

- **Ámbito:** supervisión transversal. Asegura que los demás roles estén haciendo su trabajo.
- Puede ver (read-only): informes de seguimiento de leads, ventas, cotizaciones, postventa, garantías, OPs en cada estación, pipeline de cobros y pagos, marketing assets publicados.
- Puede comentar: dejar notas/comentarios sobre leads, cotizaciones, pedidos, OPs, garantías, postventas (modelo de comments con autor + timestamp). Los comentarios son visibles al rol asignado al ítem.
- No puede: crear, editar, eliminar ni cambiar estado de ninguna entidad operativa o contable.
- Diferencia con Auditoría: Auditoría interviene operativamente (cierra OPs, edita productos); Subauditoría solo observa y comenta.

HR (Recursos Humanos)

- **Ámbito:** módulo RRHH (15.3.18) y entrada de datos a Nómina.
- **Puede:** crear/editar/desactivar empleados (nombre, rol, área, salario, fecha contratación, estado activo/inactivo); cargar novedades de nómina (horas extras, ausencias, incapacidades) que alimentan runPayroll; ver el preview de cálculo de nómina antes de contabilizar.
- **No puede:** contabilizar nómina (esa acción la dispara contabilidad o admin), ni editar los parámetros tributarios de nómina (PAYROLL_2026 — eso lo hace contabilidad en Perfil tributario).
- **Puede ver:** solo módulo RRHH y vista previa de nómina del mes en curso.

9.3. Subrol Contabilidad-Tesorería

Tesorería es un **subrol que hereda de contabilidad** pero con permisos restringidos. Se modela como `role='contabilidad' && subrole='tesoreria'` en el profile del usuario.

- **Puede:** gestionar bancos (CRUD de cuentas, ajustes manuales que disparan postBankAdjustment); aprobar pagos entrantes (approvePayment → postCustomerCollection); confirmar pagos salientes (confirmOutgoingPayment → postSupplierPayment); ver Cartera, CxP, conciliación bancaria, saldos de bancos.
- **Puede ver** (heredado de contabilidad): Balance general, P&G, libro diario y mayor (read-only).
- **No puede:** emitir facturas/NC/ND, tocar PUC ni mapeos, cerrar/reabrir periodos, correr nómina, modificar reglas tributarias.

9.4. Permisos delegables (Admin override)

Mecanismo nuevo en v1.1: cualquier usuario con rol admin/gerencia puede otorgar permisos puntuales a cualquier usuario de cualquier otro rol, sin cambiar su rol base.

- **Modelo:** tabla `user_permission_grants` en Supabase con (userId, permissionKey, grantedBy, grantedAt, expiresAt?).
- **PermissionKeys** (catálogo extensible): `can_close_period`, `can_reopen_period`, `can_emit_nc_nd`, `can_edit_puc`, `can_run_payroll`, `can_approve_payment`, `can_edit_orders`, `can_edit_products`, `can_view_all_reports`, etc.
- La política RLS de Supabase consulta primero el rol base; si el rol no permite la acción, consulta `user_permission_grants` para ese userId + permissionKey vigente.
- **UI:** pantalla "Admin → Permisos" donde un admin selecciona un usuario, ve sus permisos heredados por rol y puede agregar/revocar grants individuales. Cada cambio queda auditado (`postedBy = admin.uid`).
- **Caso de uso típico:** comercial al que se le delega `can_view_all_reports` por un sprint; tesorero al que se le delega temporalmente `can_emit_nc_nd`.

9.5. Implementación técnica

- `state.currentUser`: {id, email, role, subrole?, name}. Rol y subrol vienen de la tabla profiles tras login.
- `state.currentUser.grants`: array de permissionKeys hidratado en login desde `user_permission_grants`.
- **Helper can(action)** en frontend: encapsula la lógica rol + subrol + grants. Cada handler de acción la consulta antes de invocar el hook.
- **RLS por tabla:** políticas escritas con la función `has_permission(auth.uid(), 'permission_key')` que combina rol + grants.
- El motor contable existente ya respeta los roles principales (closePeriod / reopenPeriod). El MVP debe extender las políticas de Postgres y la UI para reflejar todos los roles nuevos.

10. Criterios de aceptación

10.1. Criterios automatizados

- ✓ node Cphora/_run_tests_node.js → 40/40 OK
- ✓ node Cphora/_smoke_e2e.js → 76 OK · 0 fallas
- ✓ node Cphora/_smoke_real_state.js → 34 OK · 0 fallas
- ✓ npm run test (frontend, cuando exista) → cobertura > 70% en motor contable
- ✓ npm run build → sin errores ni warnings críticos
- ✓ npm run lint → sin errores

10.2. Criterios funcionales (revisión manual)

- ✓ Login con email/password funciona
- ✓ Logout limpia sesión y redirige a login
- ✓ Crear lead → convertirlo en cliente → cotización → pedido → OP → entregar → facturar → cobrar
→ contabilizar genera todos los JEs esperados (verificable en Libro Diario)
- ✓ Emitir NC sobre factura entregada reversa venta + COGS correctamente
- ✓ Correr nómina del mes contabiliza y deja el banco con el neto a pagar
- ✓ Editar parámetros de nómina (SMMLV, ARL, %) los persiste y aplica en próxima corrida
- ✓ Crear/editar/eliminar cuenta bancaria funciona (eliminación bloqueada si hay movimiento)
- ✓ Mover producto terminado a bodega Insumos 1 es rechazado por validación de tipo
- ✓ Admin otorga permiso can_emit_nc_nd a usuario tesorería y este puede emitir NC
- ✓ Auditor crea y cierra OP; subauditor solo puede ver y comentar
- ✓ Exportar a FomPlus produce archivo válido para integración
- ✓ Backup desde DevTools y restore desde un JSON pegado funciona
- ✓ Subir foto de garantía a Storage y verla en el detalle

10.3. Criterios de calidad

- ✓ Cero errores en consola del navegador en flujo feliz
- ✓ Cero warnings de Tailwind/React no resueltos
- ✓ Cero queries N+1 en reportes (usar joins o `select` con relaciones de Supabase)
- ✓ Documentación actualizada (README, deployment, runbook)
- ✓ Variables de entorno documentadas en .env.example
- ✓ CI verde en main

11. Entregables esperados

Al cierre del MVP el programador debe entregar:

- Repositorio Git con el código fuente del frontend + scripts de migración + tests.
- Documentación en el repo: README.md (setup local en 10 pasos), DEPLOYMENT.md (deploy a producción), RUNBOOK.md (operaciones comunes: correr migración, hacer backup, restaurar, agregar usuario), ARCHITECTURE.md (diagramas y decisiones de diseño).
- Proyecto Supabase configurado con schema, RLS, vistas, RPCs, buckets de Storage, primer admin user.
- Deploy en producción (URL accesible, dominio si Castor lo provee).
- CI/CD configurado en GitHub Actions (o equivalente) corriendo en cada push a main.
- Tests pasando: 40 unit + 3 smokes + tests nuevos del frontend.
- Demo en vivo con el dueño del negocio (Alexander Vivas) para walkthrough completo del MVP.
- Plan de mantenimiento sugerido (cuánto soporte requiere Castor mensualmente).

12. Hitos sugeridos

Semana	Hito	Salida verificable
1	Setup + arquitectura + auth	Login funciona; tablas Supabase aplicadas; CI corre tests
2	Catálogo maestro + clientes/leads	CRUD de productos, clientes, leads, proveedores con UI completa
3	Cotizaciones + pedidos + OPs	Flujo lead → cotización → pedido → OP con UI
4	Inventario + compras + OCs	Almacén funcionando (validación PT/MP); OCs con recepción parcial
5	Pagos + bancos (CRUD) + facturación	Cobros + emisión factura/remisión con JEs; bancos editables
6	Nómina (editable) + activos fijos + cierres	Procesos batch + cierres mensuales; parámetros editables
7	Reportes + exportador FomPlus + libros PDF	Hub de exportación funcionando
8	RLS + roles nuevos + permisos delegables + Realtime	Matriz de §9 implementada; admin puede delegar permisos
9	Garantías + postventa + marketing + innovación + auditoría + subauditoría + HR	Módulos secundarios y roles completos
10	Pulido + UAT con Alexander + migración del state real	Demo aceptado

Total estimado: 10 semanas para un senior full-stack dedicado. Estos son hitos sugeridos por el equipo, no compromiso del programador. Solicitar estimación propia.

13. Riesgos y mitigaciones

Riesgo	Probabilidad	Impacto	Mitigación
Frontend tarda más de lo estimado por la cantidad de vistas (25+)	Alta	Alto	Priorizar las 5 vistas críticas (Pedidos, OPs, Facturación, Cartera, Libro Diario) en las primeras 4 semanas. Resto en paralelo.
Re-escritura del motor contable	Media	Crítico	No reescribir. Cargar castor_accounting.js tal cual. Si hay incompatibilidades con el framework, adaptar el wrapper, no el motor.
Migración del state real falla por datos sucios	Media	Alto	Probar la migración con el state real ANTES del deploy. Los filtros validOrderId/validPedidoId cubren los casos conocidos post-V7.
RLS rompe queries del frontend	Media	Medio	Diseñar las políticas antes de implementar UI. Cada feature debe validar contra los 11 roles.
Performance de reportes grandes	Baja	Medio	Usar las vistas pre-computadas (v_account_balance, v_balance_general) en vez de calcular desde journal_lines.
Confusión entre Pedidos y OPs (estructura V7)	Alta	Bajo	Leer migrateLegacy() en Demo6 antes de implementar — separación clara.

Riesgo	Probabilidad	Impacto	Mitigación
Numeración interna se descoordina con DIAN cuando se conecte facturación electrónica	Media (futuro)	Bajo	Documentar claramente que la numeración actual es NO oficial DIAN.
Cambio de exportadores impacta integraciones	Baja	Medio	FomPlus es el único exportador soportado en v1.1. Confirmar especificación de FomPlus con el proveedor antes de cerrar formato.

14. Apéndices

14.1. Lista de archivos clave (fuera de este PDF)

Cphora/Castor_Dashboard_Demo6.html	→ UX y flujos. Lee el `seed()` y los handlers.
Cphora/castor_accounting.js	→ Motor contable. NO reescribir, reutilizar.
Cphora/supabase_schema.sql	→ Schema base v1.
Cphora/supabase_schema_v2.sql	→ Adiciones.
Cphora/supabase_state_blob.sql	→ Tabla state_blobs.
Cphora/_supabase_analytics_views.sql	→ Vistas + RPC.
Cphora/_migrate_to_supabase.js	→ Generador de SQL inserts.
Cphora/_migration_castor.sql	→ SQL listo para pegar.
Cphora/_run_tests_node.js	→ 40 unit tests.
Cphora/_smoke_e2e.js	→ 76 checks E2E.
Cphora/_smoke_real_state.js	→ 34 checks contra seed real.
Cphora/README_ACCOUNTING.md	→ Manual técnico del motor contable.
Cphora/CHANGELOG_ACCOUNTING.md	→ Historia + bugs arreglados.
Cphora/DEPLOYMENT_GUIDE.md	→ Despliegue a Supabase paso a paso.
REQUERIMIENTOS_MVP.md	→ Este documento.

14.2. Decisiones tomadas (no re-discutir sin razón fuerte)

- BD: Supabase Postgres. Definitivo.
- Motor contable: castor_accounting.js reutilizado. No reescribir en Python/TS/Go.
- Mantener arquitectura cliente-pesado para hooks contables. Los cálculos corren en el navegador del usuario, no en el servidor. El servidor solo persiste.
- No emisión electrónica DIAN en el MVP. Es un proyecto separado.
- Numeración interna con leyenda "No oficial DIAN" visible en cada secuencia.
- Idempotencia por (source, sourceId) en todos los hooks. No tocar.
- PUC colombiano clases 1-7. No internacionalizar.
- COP única moneda.
- Idioma único: español Colombia.
- Tailwind para estilos. No Bootstrap, no Material UI.
- **FomPlus es el único exportador ERP soportado (v1.1).** Los anteriores (Siigo, World Office, Helisa, Contapyme) quedaron descartados.
- **Bodegas con tipo estricto PT vs MP (v1.1).**
- **Bancos default: Bancolombia, BBVA, Bold, Lulo Bank con CRUD (v1.1).**
- **Parámetros de nómina editables desde UI (v1.1).**

14.3. Preguntas frecuentes anticipadas

Q: ¿Por qué el motor contable está en un único archivo de 7.259 líneas?

A: Decisión consciente. Es un único módulo cohesivo, sin dependencias, fácil de cargar como `<script>` y de testear en aislado. No fragmentar sin necesidad.

Q: ¿Puedo cambiar la estructura de la BD si encuentro algo mejor?

A: Sí, con dos condiciones: (1) el motor contable sigue funcionando sin cambios; (2) la migración desde el seed actual sigue siendo posible y verificable.

Q: ¿Cómo manejo las fotos / soportes de pago / garantía?

A: Supabase Storage. Buckets pagos/, garantias/, marketing/. Política RLS: solo authenticated. URL firmada para descarga.

Q: ¿Tengo que mantener el sync blob de state_blobs?

A: Sí, como respaldo total durante el MVP. En una segunda fase se puede retirar cuando la sincronización por entidad esté completa para todas las tablas.

Q: ¿Cómo pruebo localmente sin Supabase real?

A: Supabase tiene CLI con Docker para correr Postgres + Studio local. Documentar en README. Como fallback, el cliente puede operar 100% en localStorage (el demo prueba esto).

Q: ¿Hay tests visuales / E2E del frontend?

A: No existen — son del motor contable nuevo. El programador debe agregarlos (Playwright o Cypress recomendado).

Q: ¿Quién es Alexander Vivas?

A: Gerente Comercial de Castor. Punto de contacto operativo para UAT y dudas de negocio. Otros stakeholders: Yolanda Reyes (contabilidad), Nelson Díaz (taller), Mario Estrada (almacén).

Q: ¿Cómo se otorgan permisos puntuales sin cambiar el rol del usuario?

A: Ver §9.4 — el Admin entra a "Admin → Permisos", selecciona el usuario, y agrega/revoca permissionKeys individuales. Cada grant queda auditado con grantedBy + grantedAt.

Q: ¿Por qué se eliminaron Siigo, WO, Helisa y Contapyme como exportadores?

A: Decisión de Castor para v1.1. FomPlus es el sistema con el que se va a integrar la contabilidad externa. El código existente de los otros exportadores en `castor_accounting.js` puede dejarse como código muerto.

14.4. Convenciones de código

- JavaScript/TypeScript: estricto ("use strict", TS estricto). 2 espacios indent.
- Naming: camelCase para variables/funciones JS, snake_case para columnas SQL. Mapeo explícito en helpers (`_jeToRow`, etc.).
- Comentarios: en español (consistente con el motor existente).
- Commits: Convencionales (feat:, fix:, refactor:, chore:).
- PRs: descripción + screenshots + checklist de criterios de aceptación tocados.

14.5. Contacto

- Dueño del producto: Fredy (cliente final).
- Stakeholders operativos: Alexander Vivas (gerencia), Yolanda Reyes (contabilidad).
- Repositorio: a definir (recomendado: GitHub privado de Castor SAS).
- Comunicación: a definir (Slack, WhatsApp, email).

Checklist para el programador antes de empezar

- ✓ Leí este documento completo (especialmente §7.5, §7.8, §7.11, §7.15 y §9 actualizados)
- ✓ Cloné el repo y abrí Cphora/Castor_Dashboard_Demo6.html en el navegador
- ✓ Recorrí los 25+ módulos del sidebar y entendí qué hace cada uno
- ✓ Leí Cphora/README_ACCOUNTING.md y entendí el motor contable
- ✓ Corrí los 3 smokes en Node y los vi en verde
- ✓ Creé un proyecto Supabase y apliqué los 3 schemas SQL
- ✓ Corrí node Cphora/_migrate_to_supabase.js y apliqué el SQL generado
- ✓ Verifiqué con las queries de DEPLOYMENT_GUIDE.md §1.3 que la migración es correcta
- ✓ Propuse stack final (con justificación) y obtuve aprobación
- ✓ Estimé el cronograma y obtuve aprobación
- ✓ Confirmé entender los 11 roles del §9 y el mecanismo de permisos delegables
- ✓ Tengo acceso a un canal de comunicación con el cliente

15. Apéndice — Inventario funcional exhaustivo Demo6

Esta sección lista TODO lo que está construido en Cphora/Castor_Dashboard_Demo6.html para que el programador lo use como checklist 1:1. Cada subsección refleja código real del demo (validado al 2026-05-09). El programador debe abrir el demo en el navegador mientras lee esta sección.

15.1. Layout global y elementos compartidos

Elemento	Detalle
Tema	Dark mode. Brand bg #0A1628, paneles #152943, gold accent #C9A961, muted #8A9BB8. Tailwind config inline en .
Tipografía	Inter (Google Fonts), 300-800.
Sidebar	256px fijo, scroll vertical, sticky top. Logo "C" + título "CASTOR" + subtítulo "Cphora · Demo 3". Footer con usuario (Alexander V.) + rol (Gerencia) + botón "Reset Data Demo".
Header sticky	Breadcrumb dinámico + título de página + búsqueda global (input 288px placeholder "Buscar...") + botón notificaciones (badge red con count, dropdown 320px) + botón "+ Nuevo" (gold).
Área central	#content-area 100% width, padding 32px, scroll vertical. Renderizada por PAGE_RENDERERS[currentPage]().
Slide-panel	560px ancho fijo, side-right, animado, fondo backdrop oscuro. #slide-panel + #slide-backdrop. Cierra con click fuera.
Modal root	#modals-root. Modales con class modal.open (display:flex). showModal(id, html) / openModal(id) / closeModal(id).
Toast root	#toast-root bottom-right. toast(msg, type) con tipos info ok warn error. Animación slideUp.

Para el inventario detallado de todas las pantallas (15.3.1 a 15.3.18), modales, slide-panels y handlers, consultar el demo HTML directamente — el programador debe navegar Demo6.html mientras implementa cada vista.

Nota: el detalle exhaustivo de cada una de las 18 vistas del Demo6 se mantiene idéntico a v1.0 del brief. Esta v1.1 solo modifica las secciones que cambian funcionalmente (7.5, 7.8, 7.11, 7.12, 7.15 y 9). El programador debe consultar Demo6.html como fuente de verdad para los detalles de UI.

2

Manual técnico del Motor Contable PUC

Módulo Contable PUC · Castor

Resumen

Motor contable embebido en el dashboard de Castor que registra automáticamente cada operación del ERP (ventas, compras, cobros, pagos, producción, nómina, depreciación, garantías) en asientos de doble partida sobre el Plan Único de Cuentas colombiano. Está pensado para que la fábrica pueda operar día a día sin contador externo y entregar al final del mes un libro listo para alimentar FomPlus. Resuelve la brecha entre la operación logística-comercial del demo y la contabilidad formal sin reescribir el frontend.

Arquitectura

El módulo es un único archivo monolítico `Cphora/castor_accounting.js` (7.259 líneas) que se carga desde `Castor_Dashboard_Demo6.html` mediante una etiqueta `<script src="castor_accounting.js"></script>` al final del body. Todas las funciones (motor, hooks, renderers UI, exportadores y suite de tests) se exportan a `window` al final del archivo. La persistencia se apoya en el state global del dashboard: el módulo añade nuevas claves (`pucAccounts`, `journalEntries`, `journalLines`, `taxRules`, etc.) y reaprovecha `save()` para volcar a `localStorage`. La función `migrateAccountingV1()` se ejecuta al cargar y siembra los catálogos por defecto si no existen, además de generar retroactivamente asientos para datos seeds previos.

API pública

Seeds

Función	Qué hace
<code>seedPucCatalog()</code>	Devuelve ~120 cuentas PUC (clases 1-7) con <code>code</code> , <code>level</code> , <code>parentCode</code> , <code>nature</code> , <code>requiresThirdParty</code> , <code>requiresCostCenter</code> , <code>taxRelated</code> . Incluye excepciones (4175, 1592, 240810).
<code>seedAccountingMappings()</code>	Diccionario operación→cuentas PUC. Define <code>sale_invoice</code> , <code>customer_collection</code> , <code>supply_purchase</code> , <code>payroll</code> , <code>depreciation</code> , etc.
<code>seedTaxRules()</code>	Reglas tributarias: 3 IVAs (19/5/0), 3 ReteFte (compras 2.5%, servicios 3.5%, honorarios 11%), ReteIVA 15%, ReteICA Bogotá 9.66×1000.
<code>seedDocNumbering()</code>	Secuencias internas (FAC, REM, NC, ND) marcadas como "no oficial DIAN".
<code>seedCostCenters()</code>	6 centros: CEDI, TALLER, VENTAS, ADMIN, CTG, VOL.
<code>seedFiscalPeriods()</code>	Periodos 2026-01..04. Los tres primeros precerrados como historial.

Motor

Función	Qué hace
<code>postJournalEntry(payload, target)</code>	Valida partida doble, catálogo, periodo, terceros y posta al libro. Genera JE-XXXXXX.

Función	Qué hace
<code>reverseJournalEntry(jeId, reason, target)</code>	Crea asiento espejo (DB↔CR) y marca el original como reversed.
<code>findJournalBySource(source, sourceId, target)</code>	Devuelve el JE posted con esa pareja (null si no hay). Es la base de la idempotencia.

Hooks por evento (13)

Función	Asiento PUC
<code>postSale(invoice)</code>	DB 130505 / CR 412035 + CR 240805 (factura con IVA inclusive); para remisión: DB 130505 / CR 412035 sin IVA. Dispara <code>postCostOfSale</code> si hay <code>orderId</code> .
<code>postCostOfSale(orderId)</code>	DB 612035 / CR 143005 al costo real registrado en <code>finishedStock</code> .
<code>postCustomerCollection(payment)</code>	DB 1110-XX / CR 130505 (o 280505 si anticipo). Si el cliente es retenedor: DB 135515/135517/135518 (anticipos de retención recibida).
<code>postCustomerAdvance(payment)</code>	Alias de collection con <code>type='anticipo'</code> y sin <code>invoiceId</code> → CR 280505.
<code>postSupplyPurchase(receipt)</code>	DB 140505 + DB 240810 (IVA descontable) / CR 220505 + CR 236540/236701/236801 (retenciones practicadas).
<code>postSupplierPayment(outgoing)</code>	DB 220505 / CR 1110-XX si está vinculado a OC; si no, DB 529595 / CR 1110-XX.
<code>postBankAdjustment({bankId, delta, concept})</code>	Si <code>delta>0</code> : DB 1110-XX / CR 425095 (diferencia positiva). Si <code><0</code> : DB 539595 / CR 1110-XX. Inyecta el banco como tercero.
<code>postRawMaterialConsumption(issue)</code>	DB 710505 / CR 140505. Inyecta CC=TALLER si la salida apunta a una sola OP.
<code>postRawMaterialReturn(issue)</code>	Inverso: DB 140505 / CR 710505.
<code>postFinishedGoods(stock)</code>	Dos asientos: JE-A DB 141005 / CR 710505 (cierre WIP) y JE-B DB 143005 / CR 141005 (entrada PT). 141005 cierra en cero. Prioriza costo real consumido sobre estimación BOM.
<code>postWarrantyCost(warranty, costEntry)</code>	DB 519540 / CR 110510 (caja) o CR 140505 (almacén). Resuelve tercero del cliente vinculado a la garantía.
<code>postManualEntry(payload)</code>	Asiento libre validado por motor. Genera JEM-XXXX.
<code>emitNotaCredito / emitNotaDebito</code>	Ver Fase 6 abajo.

Procesos batch (Fase 4)

Función	Qué hace
<code>runPayroll(periodId, employeeIds?)</code>	Calcula nómina mes (smmlv 2026, exoneración Ley 1607, riesgo ARL III, aux transporte <2 SMMLV). Persiste en <code>state.payrollRuns</code> . Parámetros editables (v1.1) desde <code>state.companyTaxProfile.payrollParams</code>.

Función	Qué hace
postPayroll(run)	Genera asiento consolidado con devengado, provisiones, aportes empleador y pasivos. MOD usa familia 7205xx + 720570 consolidado; admin/ventas usan 5105xx/5205xx.
runMonthlyDepreciation(periodId)	Línea recta sobre state.fixedAssets activos, hasta tope cost-salvageValue. Persiste en state.depreciationRuns.
postDepreciation(run)	DB 516015/526010/730560 (según uso) / CR 159220/159225/159240/159205 (según categoría, incluyendo "otros").

Reportes y consultas (Fase 5)

Función	Qué hace
getSaldoCuenta(code, asOfDate?)	Saldo a fecha incluyendo asientos posted y reversed.
getSaldoCuentaRango(code, from, to)	{saldoInicial, debitos, creditos, saldoFinal} del rango.
getMovimientosCuenta(code, from, to)	Líneas con saldo corrido para libro auxiliar.
agingByThirdParty(prefix, asOfDate?)	Aging por tercero en cubetas 0-30/31-60/61-90/91+ días.
getSalDOSPorClase(asOfDate?)	Totales por clase 1..7 respetando ecuación contable A=P+Pat+Util.
getUtilidadAcumulada(periodOrDate)	clase 4 – (5+6+7) hasta fecha.

Cierres y NC/ND (Fase 6)

Función	Qué hace
closePeriod(periodId)	Genera asiento que traslada saldos clases 4-7 contra 360505. Bloquea nuevos asientos en el periodo. Rol requerido: gerencia / contabilidad.
reopenPeriod(periodId)	Reversa el asiento de cierre. Solo si no hay periodo posterior cerrado. Rol: gerencia.
closeFiscalYear(year)	Cierra los 12 meses en cascada y traslada 360505 → 370505.
emitNotaCredito(invoiceId, motivo, items[])	DB 417505 + DB 240805 / CR 130505 al monto NC. Si hay COGS asociado, genera reverso parcial DB 143005 / CR 612035 al ratio.
emitNotaDebito(invoiceId, motivo, items[])	DB 130505 / CR 412035 + CR 240805 (incremento al cobro).
renderLibroPDF(kind, periodId)	Genera PDF con marca de agua "DOCUMENTO INTERNO · NO OFICIAL DIAN" para 'diario' 'mayor' 'inventarios' 'balances'. Fallback texto si no hay jsPDF.

Exportadores (v1.1)

Cambio v1.1: el motor mantiene los 4 exportadores ERP previos en su código (Siigo, World Office, Helisa, Contapyme) por compatibilidad, pero el Hub de exportación de la UI **solo expone FomPlus** + los 3 genéricos.

Función	Formato	Cobertura
<code>exportToCSV(view, filters)</code>	CSV con BOM UTF-8 separador ,	Todas las vistas
<code>exportToJson(view, filters)</code>	JSON array de objetos	Todas las vistas
<code>exportToXLSX(view, filters)</code>	CSV con BOM separador ; (Excel-CO)	Todas las vistas
<code>exportFomPlusFormat(periodId)</code>	Formato propio de FomPlus para integración directa (a confirmar layout con proveedor).	journal_entries
<code>window.exportReporte(view, format, filters)</code>	Dispatcher único	Llamado desde Hub

Roles y permisos (v1.1)

El motor de cierre lee `state.currentUser.role` y consulta los grants delegados. Roles principales y permisos de cierre:

Rol	Cerrar mes	Reabrir mes	Ve reportes
gerencia (admin)	Sí	Sí	Sí (todos)
contabilidad	Sí	No	Sí
tesorería (subrol)	No	No	Sí (cartera, CxP, bancos)
auditoría	No (gestiona OPs)	No	Sí
subauditoría	No	No	Sí (read-only)
marketing / innovacion / almacen / hr / comercial / produccion	No	No	Solo su área
(con grant <code>can_close_period</code>)	Sí (vía permiso delegado)	—	—
Sin <code>currentUser</code> (modo dev)	Sí	Sí	Sí

Si el usuario no tiene rol asignado el motor permite todas las operaciones (modo dev) para migraciones iniciales sin bloquear el desarrollo.

3

Changelog del Motor Contable

Changelog · Módulo Contable PUC de Castor

Fecha de cierre técnico: 2026-05-07 · **Última corrección de brief:** 2026-05-21 (v1.1)

Las 6 fases del módulo contable están completas y validadas. El motor (`castor_accounting.js`, 7.259 líneas) cubre todo el ciclo operativo de Castor desde compra de materia prima hasta cierre fiscal anual, con 38 tests unitarios y 63 checks E2E pasando en verde. Este documento registra qué entregó cada fase y los 4 bugs que se descubrieron y arreglaron durante la validación final.

Fase 1 · Andamiaje, motor y MVP UI

Entregada: 2026-04-22

Primer commit del módulo. Se sienta toda la base sobre la que crecerán las demás fases.

- Seeds: `seedPucCatalog` con 120 cuentas (clases 1-7), `seedAccountingMappings`, `seedTaxRules` (8 reglas), `seedDocNumbering` (FAC/REM/NC/ND), `seedCostCenters` (6) y `seedFiscalPeriods` (2026-01..04).
- Motor: `postJournalEntry` con validaciones de partida doble, catálogo, terceros e idempotencia por (source, sourceId).
- Hooks núcleo: `postSale`, `postCostOfSale`, `postCustomerCollection`, `postCustomerAdvance`, `postSupplyPurchase`, `postSupplierPayment`, `postBankAdjustment`, `postManualEntry`.
- Helpers de banco: `ensureBankPucAccount` crea auxiliares 1110-XX automáticamente.
- UI MVP: subsidebar contable, dashboard con KPIs del mes, libro diario, libro mayor, auxiliar de cuenta, asientos manuales.

Fase 2 · IVA, retenciones y UI editable

Entregada: 2026-04-30 · Cierre del flujo tributario.

- Tax helpers: `calcTaxBreakdown` con modos inclusive/exclusive y soporte de descuento, IVA y retenciones.
- `postSupplyPurchase` resuelve IVA (IVA-19 default) y aplica retenciones practicadas (RTF-25/35/11, RetelIVA-15, RetelICA-Bogotá).
- `postCustomerCollection` autocalcula retenciones cuando `customer.retenedor === true`.
- UI: `renderConfigImpuestos` con CRUD de reglas.

Fase 3 · Producción, costo de venta real y centros de costo

Entregada: 2026-05-05 / 2026-05-06

- `postRawMaterialConsumption(issue)` → DB 710505 / CR 140505.
- `postRawMaterialReturn(issue)` invertido para devoluciones.
- `postFinishedGoods(stock)` genera DOS asientos: JE-A cierre WIP, JE-B entrada PT.
- `postWarrantyCost(warranty, costEntry)`.
- Centro de costo: `_resolveCostCenter` mapea las 9 estaciones de producción.

Fase 4 · Nómina y activos fijos

Entregada: 2026-05-06

- Constante PAYROLL_2026 con SMMLV \$1.423.500, aux transporte \$200.000, salud emp 4%/empr 8.5%, pensión emp 4%/empr 12%, ARL 0.522%, parafiscales SENA 2%+ICBF 3%+Caja 4%, exoneración Ley 1607.
- **v1.1:** los valores PAYROLL_2026 se exponen como defaults en `state.companyTaxProfile.payrollParams` y son totalmente editables desde UI.
- `runPayroll(periodId, employeeIds?)` calcula nómina; `postPayroll(run)` contabiliza con MOD consolidado.
- Activos fijos: `runMonthlyDepreciation` + `postDepreciation`. **v1.1:** añadida categoría "otros" en activos fijos.

Fase 5 · Reportes y hub de exportación

Entregada: 2026-05-06 / 2026-05-07

- Helpers de saldo y movimientos: `getSaldoCuenta`, `getSaldoCuentaRango`, `getMovimientosCuenta`, `agingByThirdParty`, `getSaldosPorClase`, `getUtilidadAcumulada`.
- Renderers de reportes: Balance de Prueba (8 columnas), Balance General, P&G, Cartera (aging), CxP (aging).
- Exportadores nativos: `exportToCSV`, `exportToJSON`, `exportToXLSX`.
- Exportadores ERP (originales): `exportSiigoFormat`, `exportWorldOfficeFormat`, `exportHelisaFormat`, `exportContapymeFormat`.
- **v1.1:** añadido `exportFomPlusFormat`; los 4 exportadores anteriores quedan en el código pero no se exponen en el Hub de la UI.

Fase 6 · Cierres, NC/ND y libros PDF

Entregada: 2026-05-07

- `closePeriod`, `reopenPeriod`, `closeFiscalYear`.
- `emitNotaCredito`, `emitNotaDebito` con manejo de periodos cerrados.
- `renderLibroPDF` con marca de agua "DOCUMENTO INTERNO · NO OFICIAL DIAN".

Bugs arreglados durante validación E2E (2026-05-07)

Durante la corrida final del smoke E2E (63 checks) salieron 4 desviaciones que no detectaron los unit tests porque cada uno usa una fixture aislada. El smoke ejercita el ciclo completo sobre el motor real.

1. `postCustomerCollection` — autocálculo de retención cliente-retenedor mandaba al pasivo

Causa: el código antiguo asignaba `account` explícito (236540 PRACTICADA) en vez de dejar al resolver elegir por `r.type` (135515 RECIBIDA). **Fix:** `remove account` explícito en autocálculo. Test cubre: `t_customer_collection_with_retenciones`.

2. `postFinishedGoods` — costo BOM divergía del consumo real, dejaba 710505 negativo

Causa: priorizaba `stock.totalCost` sobre el costo real consumido. **Fix:** cambiar jerarquía para priorizar consumo real; fallback a estimación BOM solo si no hay consumo previo registrado.

3. `getSaldoCuenta` y `getSaldosPorClase` — filtraban `status='posted'`, excluían reversados

Causa: tras reapertura, asientos reversed quedaban excluidos sesgando saldos. **Fix:** incluir ambos estados (posted y reversed) que suman a cero entre sí.

4. postBankAdjustment — cuentas 425095/539595 requerían tercero

Causa: builder de líneas no inyectaba thirdParty en la línea de diferencias. **Fix:** inyectar el banco como tercero placeholder en ambas líneas del JE.

Próximos pasos sugeridos (no en este release)

- Conexión con proveedor de facturación electrónica DIAN.
- Auditoría de cambios en mapeos: hoy editar accountingMappings o taxRules desde UI no genera registro histórico.
- Conciliación bancaria automática: el motor expone postBankAdjustment para diferencias manuales, pero no tiene un matcher de extractos contra movimientos.
- Renta presuntiva y declaración anual: hoy closeFiscalYear traslada utilidad pero no calcula renta.
- Multimoneda: todo está en COP.
- Costos indirectos de fabricación dinámicos: hoy CIF entra solo como gastos manuales.
- **(v1.1)** Validación de tipo de bodega PT vs MP en cada movimiento de inventario.
- **(v1.1)** Implementar tabla user_permission_grants y el helper has_permission para RLS.
- **(v1.1)** Finalizar la especificación oficial del formato FomPlus con el proveedor.

4

Guía de despliegue a Supabase

Castor · Guía de despliegue del módulo contable a Supabase

Llevar el módulo contable PUC de Castor desde `localStorage` (Demo6 actual) a una base de datos Supabase real. Esta guía está dividida en fases — la primera es de bajo riesgo (Supabase como respaldo) y las siguientes incrementan progresivamente.

Pre-requisitos generales:

- Proyecto Supabase activo (<https://supabase.com>)
- Acceso al SQL Editor del proyecto
- Backup de `localStorage` actual
- Las 3 suites de validación pasando

Fase 0 — Backup obligatorio antes de cualquier cosa

Antes de tocar Supabase, exporta el state actual de `localStorage` desde el navegador donde tienes Demo6 abierto. Esto es tu seguro.

```
copy(JSON.stringify(JSON.parse(localStorage.getItem('castor_cphora_v5'))), null, 2));
```

Pega el contenido en un archivo `castor_state_backup_YYYYMMDD.json` y guárdalo fuera del proyecto (Drive, Dropbox, etc.).

Fase 1 — Schema + datos en Supabase (read-only, sin tocar Demo6)

Objetivo: tener todas las tablas contables creadas y pobladas en Supabase con el state actual. Demo6 sigue usando `localStorage` como antes — Supabase queda como **respaldo de solo lectura**.

1.1. Aplicar el schema base + v2 + state_blobs

El schema se aplica en tres scripts separados, en este orden: `supabase_schema.sql` (43 tablas: 29 plataforma + 14 contables) → `supabase_schema_v2.sql` (columnas faltantes + tablas `purchase_orders/items` + **v1.1: warehouse_type** para distinguir PT vs MP) → `supabase_state_blob.sql` (tabla auxiliar para blob sync).

1.2. Generar la migración desde tu state actual

```
cd Cphora
node _migrate_to_supabase.js
```

Esto produce `_migration_castor.sql` (~580 líneas) con los INSERT derivados del seed real, cubriendo TODAS las tablas (no solo contabilidad).

1.3. Aplicar la migración y verificar

```
-- Ecuación contable: Activo = Pasivo + Patrimonio + Utilidad
select
  (select coalesce(sum(saldo),0) from v_account_balance where class_code='1') as activo,
  (select coalesce(sum(saldo),0) from v_account_balance where class_code='2') as pasivo,
  (select coalesce(sum(saldo),0) from v_account_balance where class_code='3') as patrimonio,
  (select coalesce(sum(saldo),0) from v_account_balance where class_code='4')
  - (select coalesce(sum(saldo),0) from v_account_balance where class_code in
```

```

    ('5','6','7')) as utilidad
-- activo debe = pasivo + patrimonio + utilidad

```

Fase 2 — Sincronización Demo6 → Supabase (escritura asíncrona)

Objetivo: cada cambio en Demo6 se replica a Supabase, pero localStorage sigue siendo la fuente de verdad para lectura. **Riesgo bajo** — si Supabase falla, Demo6 sigue trabajando. Estado: implementado en código (2026-05-07). Solo falta configurar credenciales reales y aplicar Fase 1 en Supabase para activarla.

Fase 3 — Lectura desde Supabase (cutover gradual)

Objetivo: Demo6 puede traer datos desde Supabase para reconciliarse con la nube. localStorage sigue siendo la fuente de verdad para los renders (síncronos, instantáneos), pero el usuario puede pedir explícitamente refrescar desde la nube con el botón "Sincronizar desde Supabase" en el Libro Diario.

Fase 4 — Multi-usuario con RLS y Realtime

Objetivo: varios usuarios trabajan simultáneamente. Cambios de uno se ven en los otros sin recargar.

4.1. Habilitar Row Level Security (RLS)

```

-- Bloquear acceso anónimo a tablas contables
alter table journal_entries enable row level security;
alter table journal_lines    enable row level security;
alter table puc_accounts     enable row level security;

-- Política: solo usuarios autenticados pueden leer
create policy "auth read journal_entries" on journal_entries
  for select to authenticated using (true);

-- Política: contabilidad/admin/auditoria pueden escribir (v1.1)
create policy "contab write journal_entries" on journal_entries
  for all to authenticated
  using (
    exists (select 1 from profiles
      where id = auth.uid()
      and (rol in ('contabilidad','admin','auditoria')
        or has_permission(auth.uid(), 'can_write_journals'))
    );

```

v1.1: implementar la función SQL `has_permission(uid, key)` que consulta la tabla `user_permission_grants` para soportar permisos delegables (ver Doc 1 §9.4).

Apéndice D — Costos estimados de Supabase

Tier	Precio	Incluye	Suficiente para Castor?
Free	\$0	500 MB DB, 5 GB bandwidth, 50k MAU	Sí, primer año
Pro	\$25/mes	8 GB DB, 250 GB bandwidth, 100k MAU	Sí, hasta 100+ usuarios concurrentes
Team	\$599/mes	RLS audit, SOC2, dedicated support	Si Castor crece a multi-empresa

Volumen de datos estimado de Castor (10 años): journal_entries ~120k filas → ~25 MB; journal_lines ~600k filas → ~120 MB. Total: < 200 MB → cabe en Free tier por años.

Resumen ejecutivo

Fase	Tiempo estimado	Riesgo	Reversible
0 — Backup	5 min	nulo	n/a
1 — Schema + migración	30 min	bajo	sí
2 — Sync write	2-3 horas	bajo	sí
3 — Read from Supabase	4-8 horas	medio	sí
4 — Multi-user + RLS + Realtime + permisos delegables	1-2 días	alto	sí

Empieza por Fase 1 + monitorea por una semana. Si todo se ve bien, Fase 2 + 3 + 4 son progresivos sin romper Demo6 funcional.

A

Anexo · Archivos a enviar por separado

Archivos código y SQL a entregar al programador

Este PDF contiene la documentación técnica. Los archivos código, SQL y test deben enviarse por separado (ZIP del proyecto o repositorio Git). Lista completa:

Ruta relativa	Descripción	Tamaño
cphora/demos/Castor_Dashboard_Demo6.html	Prototipo funcional del dashboard (UX y comportamiento de referencia)	601.0 KB
cphora/engine/castor_accounting.js	Motor contable PUC (7259 líneas, reutilizable tal cual)	352.2 KB
cphora/db/supabase_schema.sql	Schema base v1 (43 tablas)	27.2 KB
cphora/db/supabase_schema_v2.sql	Adiciones aditivas v2 (purchase_orders + columnas faltantes + warehouse_type v1.1)	9.7 KB
cphora/db/supabase_state_blob.sql	Tabla auxiliar state_blobs para blob sync	1.0 KB
cphora/db/_supabase_analytics_views.sql	Vistas BI + RPCs (Balance, P&G, KPIs, cash flow)	16.8 KB
cphora/db/_migrate_to_supabase.js	Migrador Node: state → SQL inserts	41.3 KB
cphora/db/_migration_castor.sql	SQL listo para aplicar (regenerado por el migrador, 577 líneas)	55.6 KB
cphora/tests/_run_tests_node.js	Runner de los 40 tests unitarios	2.4 KB
cphora/tests/_smoke_e2e.js	Smoke E2E (76 checks del ciclo operativo completo)	19.9 KB
cphora/tests/_smoke_real_state.js	Smoke contra seed real (34 checks)	7.8 KB
cphora/tests/_smoke_browser.html	Smoke ejecutable en navegador	14.7 KB
cphora/db/supabase_client.js	Skeleton del cliente Supabase (no usar — cliente vive inline en Demo6)	1.2 KB

Cómo entregar el paquete

- Crear un ZIP que contenga los 13 archivos listados arriba y este PDF en la raíz.
- Alternativa preferida: crear repositorio Git privado con esa estructura; el PDF queda en la raíz del repo.
- Confirmar que el programador puede: abrir Demo6, ejecutar los 3 tests Node, leer este brief completo.
- Si confirma esos 4 puntos, tiene el 80% del contexto que necesita para empezar.
- Resto: leer este PDF completo (especialmente §15 del Doc 1 — el checklist 1:1 contra Demo6) y proponer cronograma + stack.

Contacto para dudas: Fredy (cliente final). **Stakeholders operativos:** Alexander Vivas (gerencia), Yolanda Reyes (contabilidad), Nelson Díaz (taller), Mario Estrada (almacén).

Resumen de cambios v1.1 (2026-05-21):

- §7.5 — Bodegas separadas estrictamente: CEDI, Castor 43, Castor Ctg, Volcanes solo PT; Insumos 1 e Insumos 2 solo MP.
- §7.8 — Cuentas bancarias por defecto: Bancolombia, BBVA, Bold, Lulo Bank. CRUD completo (registrar/editar/eliminar).
- §7.11 — Todos los valores de nómina (SMMLV, tasas, provisiones) son totalmente editables desde la UI.
- §7.12 — Añadida categoría "Otros" en activos fijos.
- §7.15 — Único exportador ERP: FomPlus. Eliminados Siigo, World Office, Helisa y Contapyme del Hub de UI.
- §9 — Capítulo completamente reescrito: añadidos roles Marketing, Innovación, Almacén, Auditoría, Subauditoría, HR; subrol Contabilidad-Tesorería; mecanismo de permisos delegables por admins.